CS 550 Operating Systems
Spring 2026

CPU scheduling

Topics

- Fundamentals of process scheduling
  - Workload, performance metrics, …
- Basic process scheduling algorithms
  - FIFO, SJF, STCF, RR
- Realistic (more advanced) process scheduling algorithms
  - MLFQ

Scheduling

- Low-level mechanisms of running processes
  - process dispatching mechanism
- High-level policies – scheduling policies
  - Scheduling appears in many disciplines (e.g., assembly lines for efficiency)
- How to develop a **framework** for studying scheduling policy?
  - assumptions of workload
  - metrics of performance
  - approaches

Workload assumptions

- Workload – {processes} running in the system
  - Critical to building policies - the more you know, the more fine-tuned your policy can be
  - Unrealistic, initially, but will be relaxed later
    1. Each job runs for the same amount of time
    2. All jobs arrive at the same time
    3. All jobs only use CPU (*i.e.*, they perform no I/O)
    4. The run-time of each job is known – scheduler know everything!

Scheduling metrics

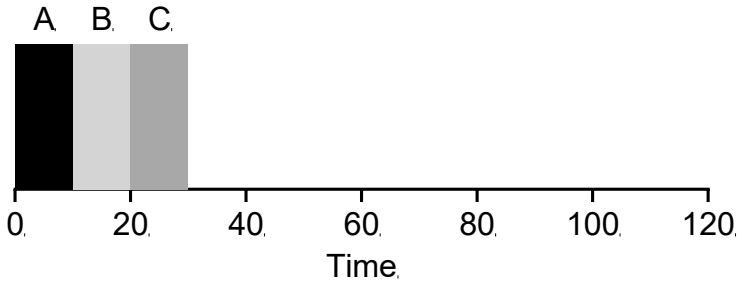
- To compare different scheduling policies
- Metrics – used to measure something
  - Turnaround time: the time at which the job completes minus the time at which the job arrived in the system (a performance metric)

$$T_{turnaround} = T_{completion} - T_{arrival}$$

  - Fairness
  - A tradeoff between fairness and performance

First In First Out (FIFO)

- A.k.a. First come, first served (FCFS)

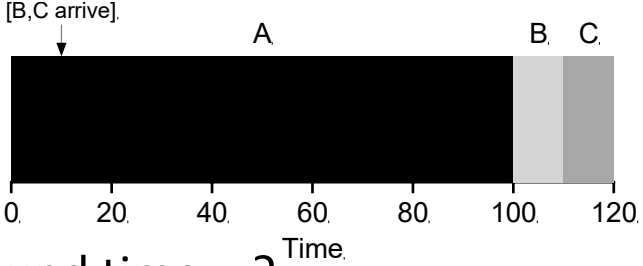


- Each job runs 10 sec.
- Average turnaround time = ?

  = (10+20+30)/3 = 20

First In First Out (FIFO)

- What kind of workload could you construct to make FIFO perform poorly?



- Average turnaround time = ?

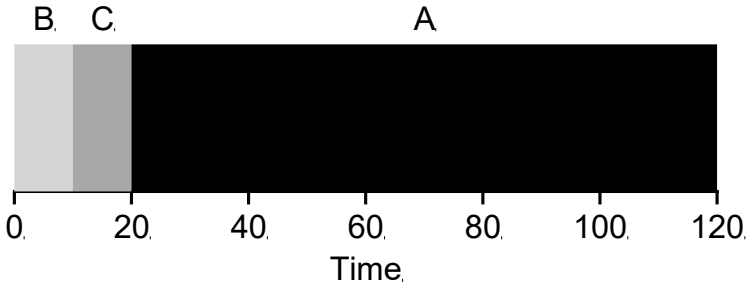
  = (100+110+120)/3 = 110

- Convoy effect
  - a number of relatively-short potential consumers of a resource get queued behind a heavyweight resource consumer

Shortest Job First (SJF)

- Average turnaround time = ?

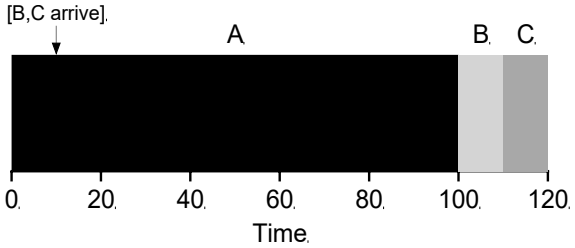
  = (10+20+120)/3 = 50



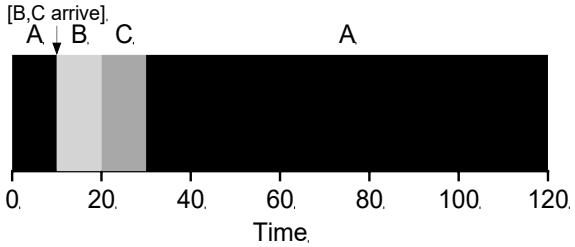
- Assuming all jobs arrive at the same time, SJF is **optimal**

Shortest Job First (SJF)

- SJF with A arrives at 0, and B, C at 10

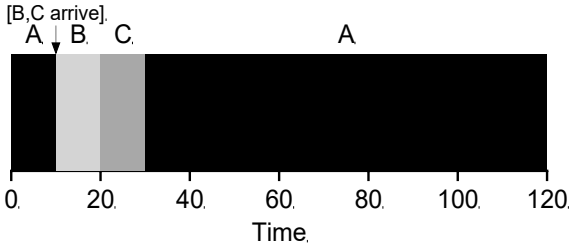


- How would you do better?
  - **Preemption**



Shortest Time-to-Completion First (STCF)

- **Shortest Time-to-Completion First (STCF)** or **Preemptive Shortest Job First (PSJF)**: any time a new job enters the system, it determines of the remaining jobs and new job, which has the least time left, and then schedules that one.



- Average turnaround time = ?

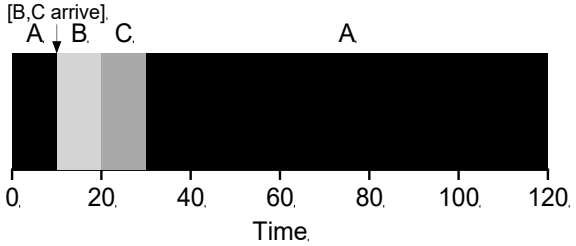
  = ((120-0)+(20-10)+(30-10))/3 = 50

A new metric: response time

- Interactive applications
- Response time = time from when the job arrives in a system to the first time it is scheduled

- Example:

$$T_{response} = T_{firstrun} - T_{arrival}$$



Average response time = ?
(0+0+10)/3 = 3.3 sec.

A new metric: response time

- STCF and related disciplines are not particularly good for response time
  - If three jobs arrive at the same time, for example, the third job has to wait for the previous two jobs to run in their entirety before being scheduled just once.
  - While great for turnaround time, STCF is quite bad for response time and interactivity.
- How can we build a scheduler that is sensitive to response time?

Round Robin (RR)

- Instead of running jobs to completion, RR runs a job for a time slice (scheduling quantum) and switches to the next job in the ready queue; repeatedly does so until jobs are finished
- Time slicing – length of time slice = multiple of timer-interrupt period
  - Three jobs, A, B, and C, whose runtimes are 5 secs, arrive in the system at time 0
    - SJF response time = ? 5 secs
    - With a time slice of 1 sec, RR response time = ? 1 sec

Round Robin (RR)

- Length of time slice on response time ?
  - The shorter the time slice is, the better the performance of RR under the response-time metric.
  - Can we make the time slice as small as possible?
  - Where do the overheads come from?
    - Context switch (saving/restoring registers), cache/TLB flushed …
  - Design trade-off: making the time slice long enough to amortize the cost of switching, but not too long that the system is no longer responsive.

Round Robin (RR)

- RR is one of the worst policies if turnaround time is the metric.
- More generally, any policy (such as RR) that is fair, (i.e., that evenly divides the CPU among active processes on a small time scale) will perform poorly on metrics such as turnaround time.
- Common tradeoff: performance (turnaround time) vs. fairness (response time)

What we discussed so far

- The basic ideas behind scheduling and developed two families of approaches.
  - The first runs the shortest job remaining and thus optimizes turnaround time.
    - Example: SJF, STCF
    - Unfortunately OS does not generally know how long a job will run for.
  - The second alternates between all jobs and thus optimizes response time.
    - Example: Round Robin
    - Algorithms like RR reduce response time but are bad for turnaround time.

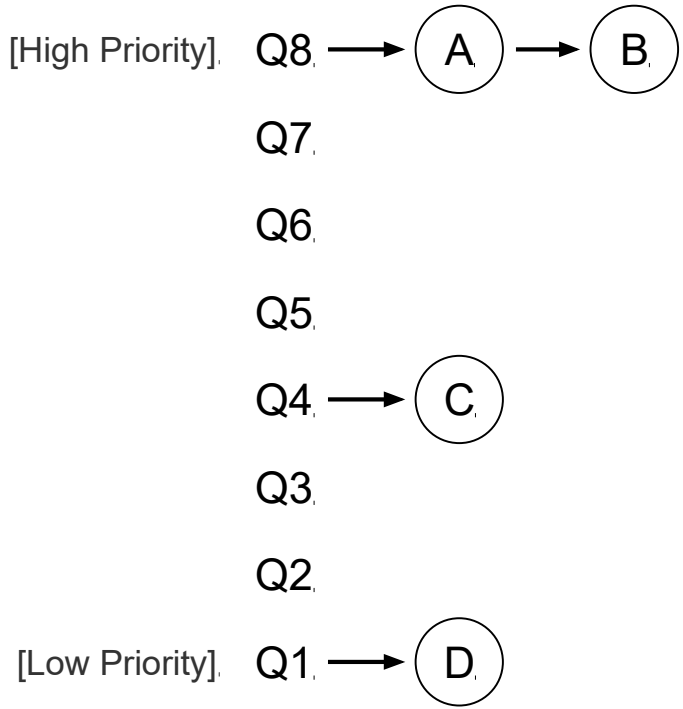
Multi-Level Feedback Queue (MLFQ)

- Developed by Turin Award winner Fernando Corbato. Adopted in many modern OSes.
- Problems to be addressed
  - optimize turnaround time (by running shorter jobs first)
  - minimize response time for interactive users
- Do we know anything (running time) about the jobs?
  - How to schedule without perfect knowledge?
  - Can the scheduler learn? How?
- **Learn from history**
  - learn from the past to predict the future

MLFQ

- Basic setup
  - The MLFQ has a number of distinct queues, each assigned a different priority level.
  - At any given time, a job that is ready to run is on a single queue.
  - MLFQ uses priorities to decide which job should run at a given time: a job with higher priority (i.e., a job on a higher queue) is chosen to run.
  - For jobs in the same queue (same priority), RR is used.
- **First two basic rules for MLFQ**
  - **Rule 1:** If Priority(A) > Priority(B), A runs (B doesn't)
  - **Rule 2:** If Priority(A) = Priority(B), A & B run in RR

MLFQ example



MLFQ

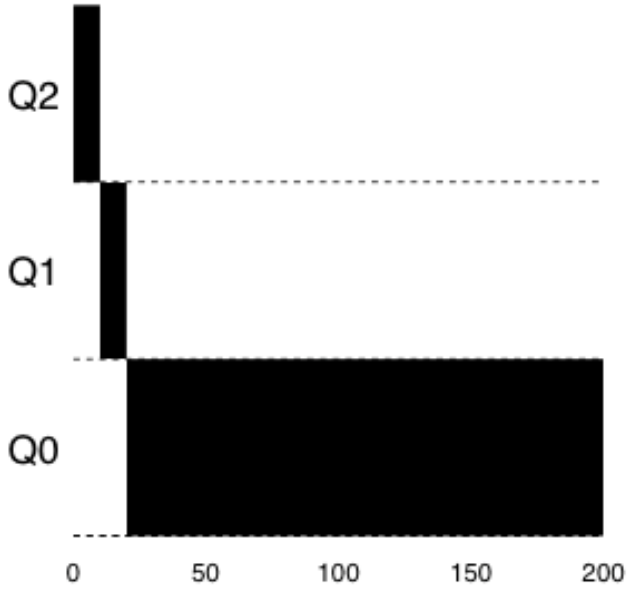
- The key to MLFQ scheduling is to properly set priorities.
- MLFQ ***varies*** the priority of a job based on its ***observed behavior***
  - Fob a job that uses the CPU intensively for long periods of time (i.e, CPU-bound job), what should MLFQ do with its priority?
    - Should reduce its priority
  - For a job that repeatedly relinquishes CPU and waits for I/O (i.e., I/O-bound job), what should MLFQ do with its priority?
    - Should keep its priority high
  - MLFQ uses *history* of a job to predict its *future* behavior

Changing priority

- **Workload** - a mix of
  - interactive jobs that are short-running (and may frequently relinquish CPU), and
  - some longer-running "CPU-bound" jobs that need a lot of CPU time but where response time isn't important
- **Rules on changing job priority**
  - **Rule 3:** When a job enters the system, it is placed at the highest priority (the topmost queue)
  - **Rule 4a:** If a job uses up an entire time slice while running, its priority is *reduced* (i.e., it moves down one queue)
  - **Rule 4b:** If a job gives up CPU before the time slice is up, it stays at the *same* priority level

Example

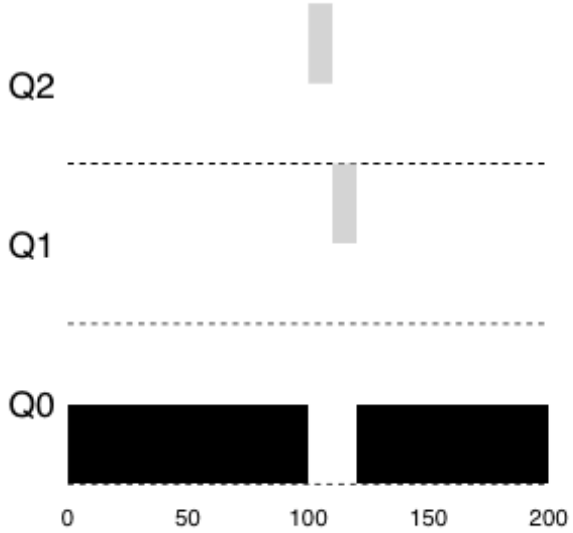
- One long-running job



Time slice length = 10 ms

Example

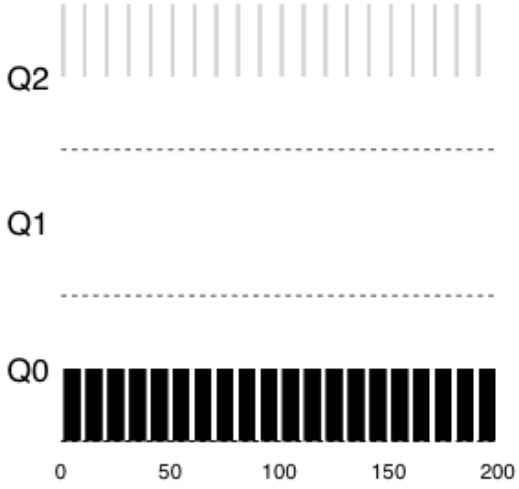
- Two jobs: A (long-running and CPU-intensive), and B (short-running and interactive)
- B arrives at T=100 ms and will run for 20 ms.
- How MLFQ approximate SJF for job B in this case?
  - A has run some time → drops to the lowest priority queue.
  - When B comes in, the scheduler doesn't *know* whether a job is short or long-running, it first *assumes* it is a short job by placing it in the highest priority queue.
  - If it actually is a short job, it will run quickly and complete; if it is not a short job, it will slowly move down the queues, and thus soon prove itself to be a long-running.



Time slice length = 10 ms

How about I/O?

- **Rule 4b:** If a job gives up CPU before the time slice is up, it stays at the *same* priority level.
  - If an interactive job, for example, is doing a lot of I/O, it will relinquish the CPU before its time slice is complete; in such case, we don't wish to penalize the job and thus simply keep it at the same level.



MLFQ

- MLFQ rules thus far
  - **Rule 1**: If Priority(A) > Priority(B), A runs (B doesn't).
  - **Rule 2**: If Priority(A) = Priority(B), A & B run in RR.
  - **Rule 3**: When a job enters the system, it is placed at the highest priority (the topmost queue).
  - **Rule 4**: If a job uses up an entire time slice while running, its priority is reduced (i.e., it moves down one queue); If a job gives up CPU before the time slice is up, it stays at the same priority level.

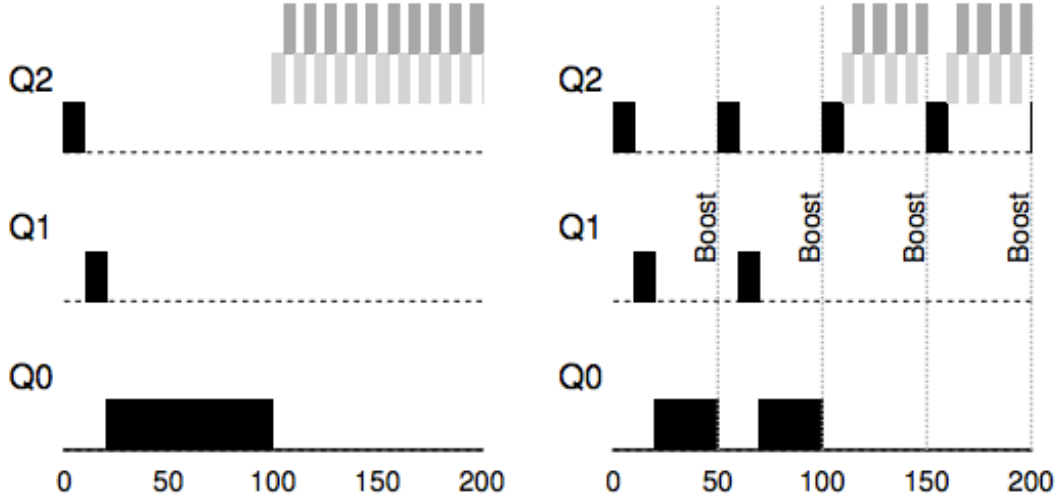
Any problems with the current version of MLFQ?

MLFQ

- Problems with the current version of MLFQ
  - Starvation
    - Too many interactive jobs, and thus long-running jobs will never receive any CPU time
  - Game the scheduler
    - Doing something sneaky to trick the scheduler into giving you more than your fair share of the resource.
  - What if a CPU-bound job turns to I/O-bound
    - There is no mechanism to move the job up to queues with higher priorities!

Boosting priority

- **Rule 5**: After some time period S, move all the jobs in the system to the topmost queue.
- What problems does this rule solve?
  - Starvation
  - When a CPU-bound job turns to I/O bound.

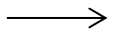


How to deal with those gamer processes?

CPU time accounting

- The scheduler keeps track of how much time a job used at a given level (i.e., in a given queue); once a job has used its allotment, it is demoted to the next priority queue.

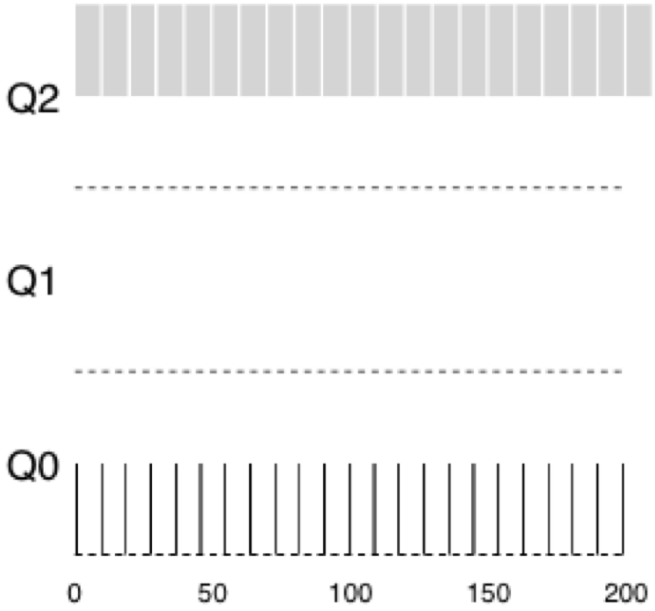
Rule 4: If a job uses up an entire time slice while running, its priority is reduced (i.e., it moves down one queue); If a job gives up CPU before the time slice is up, it stays at the same priority level.



Rule 4: Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e., it moves down one queue).

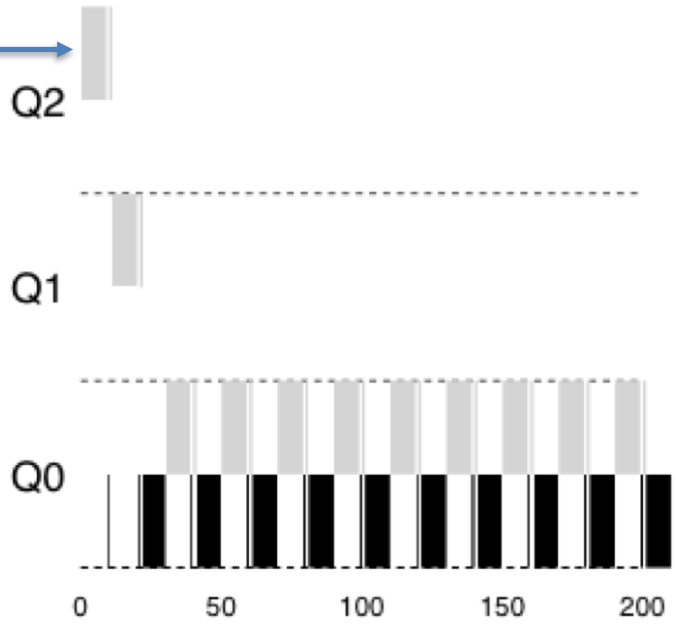
CPU time accounting

Before (with the old rule 4)



The light shadow indicates how the gamer process uses the CPU

After (with the new rule 4)



Tuning MLFQ

- How to parameterize MLFQ ?
  - How many queues?
  - How big should time allotment be per queue?
  - How often should priority be boosted?
- No easy answers and need experience with workloads
  - Varying time allotment length across different queues (e.g., high-priority queues are usually given short time allotment, and low-priority queues are with long time allotment)
  - Some schedulers reserve the highest priority levels for operating system work.
  - Some systems also allow some user advice to help set priorities (e.g., for example, by using the command-line utility `nice` you can increase or decrease the priority of a job (somewhat) and thus increase or decrease its chances of running at any given time).

MLFQ summary

- **Rule 1:** If Priority(A) > Priority(B), A runs (B doesn't)
- **Rule 2:** If Priority(A) = Priority(B), A & B run in RR
- **Rule 3:** When a job enters the system, it is placed at the highest priority (the topmost queue)
- **Rule 4:** Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced (i.e., it moves down one queue)
- **Rule 5:** After some time period S, move all the jobs in the system to the topmost queue
- Instead of demanding *a priori* knowledge of a job, MLFQ instead observes the execution of a job and prioritizes it accordingly
- MLFQ manages to achieve the best of both worlds: it can deliver excellent overall performance (similar to SJF/STCF) for interactive jobs, and is fair and makes progress for long-running CPU-intensive workloads